



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

WRITTEN ASSESSMENT (SAQ) — ANSWER KEY

For



Agentic AI Automation with n8n

TGS Ref No: TGS-2023035977

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version v5

Agentic AI Automation with n8n

Answers to Written Assessment (SAQ)

Course Code: TGS-2023035977

Short-Answer Questions (Knowledge)

Answer all questions in your own words. Each question tests underpinning knowledge covered in the course slides.

Question 1:

Generative AI systems are typically designed to produce content in response to user prompts, while agentic AI systems are built to plan, decide, and take actions autonomously to achieve goals over multiple steps.

What are the key differences between Generative AI and Agentic AI in terms of capabilities, behaviour and use cases? (K1)

Suggestive answers (not exhaustive):

- Autonomy: Generative AI responds to a prompt; Agentic AI plans, decides and acts on its own.
- Goals: Generative AI handles a single task; Agentic AI pursues a multi-step goal.
- Tools/memory: Agentic AI calls tools/APIs, keeps memory and looks up data (RAG); Generative AI does not.
- Use cases: content generation vs. autonomous, multi-step workflows (e.g. a Telegram support agent).

Question 2:

In n8n, workflows can be started in different ways depending on how and when automation should occur — by a user action, a time-based event, or a request from an external system.

What are the key trigger nodes in n8n that can be used to start a workflow? (K2)

Suggestive answers (not exhaustive):

- Manual Trigger
- Form Trigger (form submission)
- Schedule Trigger (time-based)
- Webhook (external HTTP request)
- Telegram Trigger / Chat Trigger (incoming message)

Question 3:

After a workflow is triggered, action nodes process data, apply logic and control the flow — enabling decision-making, branching, data handling and custom logic.

What are some key action / logic nodes in n8n used to control workflow logic and data processing? (K3)

Suggestive answers (not exhaustive):

- IF node (branching on a condition)
- Switch node (multi-way routing)
- Merge node

- Edit Fields (Set) node
- Code node (JavaScript/Python)
- Filter node

Question 4:

An AI agent perceives context, reasons about a task and takes actions autonomously to achieve a goal. To work, it must combine reasoning with memory and the ability to use external systems.

What are the key components required to build a functional AI agent in n8n? (K4)

Suggestive answers (not exhaustive):

- A Chat Model (LLM) for reasoning — e.g. OpenAI or Google Gemini.
- Memory (e.g. a window buffer) to retain conversation context.
- Tools (Vector Store, Data Table, HTTP, sub-workflows) the agent can call.
- A System Instruction that defines the agent's scope, rules and how to route between tools.

Question 5:

You are building an AI system that must retrieve relevant information from your own documents and use that context to generate accurate, up-to-date answers with an LLM — improving factual grounding and reducing hallucination without retraining the model.

Which type of AI workflow best supports this combination of information retrieval and language generation, and how does it work? (K5)

Suggestive answers (not exhaustive):

- Retrieval-Augmented Generation (RAG).
- Documents are split into chunks, embedded, and stored in a vector store.
- At query time the question is embedded and the closest chunks are retrieved.
- The retrieved chunks are given to the LLM as context so it answers only from your documents.

Question 6:

You are designing an n8n workflow that must fetch data from external systems and online services (e.g. a market-data or news API) and use the returned data in later steps.

Which method would you use in n8n to connect to external data sources and services, and what do you configure on it? (K6)

Suggestive answers (not exhaustive):

- The HTTP Request node (or a built-in service node).
- Configure the method (GET/POST), URL, headers and query parameters.
- Store API keys in credentials (e.g. Query Auth / header auth) — never hard-coded.
- Use the structured JSON response in downstream nodes.

Question 7:

In a RAG system, text must be turned into a form that supports meaning-based (semantic) search before it can be stored and retrieved.

Explain how embeddings and a vector store enable semantic retrieval in a RAG pipeline. (K7)

Suggestive answers (not exhaustive):

- An embedding turns a chunk of text into a vector (a list of numbers capturing meaning).
- Similar meanings produce vectors that are close together.
- Vectors are saved in a vector store; the query is embedded and the nearest vectors are retrieved (e.g. cosine similarity).
- Vector indexing keeps similarity search fast as the dataset grows.

Question 8:

You try to upload documents into a vector store for RAG, but the ingestion fails or the vectors are rejected. The store was already initialised with a specific embedding configuration.

What is the most likely underlying issue to check first, and what is the rule? (K8)

Suggestive answers (not exhaustive):

- Embedding-dimension mismatch.
- The embedding model's output dimension must equal the vector store's configured dimension.
- e.g. OpenAI text-embedding-3-small = 1536 dimensions → the index/table/collection must be 1536.
- Changing the embedding model changes the dimension (e.g. -3-large = 3072), so recreate the store to match.

Question 9:

An in-memory vector store is simple but is lost when the workflow restarts. For production RAG you want a store that persists and scales.

Name persistent vector databases you can use for RAG in n8n, and one distinguishing trait of each. (K9)

Suggestive answers (not exhaustive):

- Pinecone — fully-managed SaaS; just create an index (dimension + metric), zero-ops.
- Supabase (pgvector) — Postgres + a vector extension; good if you already use Postgres.
- Qdrant — open-source; run via Docker or Qdrant Cloud for full control.
- (All must match the embedding dimension, e.g. 1536.)

Question 10:

To make an AI agent trustworthy, you place safety checks around it so that unsafe input never reaches the agent and unsafe output never reaches the user.

What is the difference between an input (pre) guardrail and an output (post) guardrail, and what does each block? (K10)

Suggestive answers (not exhaustive):

- Input / pre-guardrail: an LLM classifies each incoming message ALLOW or BLOCK before it reaches the agent.
- It blocks prompt injection, jailbreaks and requests for another person's private data.

- Output / post-guardrail: an LLM checks every reply SAFE or LEAK before it reaches the user.
- It blocks salary figures, NRIC, credentials or system-instruction leaks; a blocked case returns a safe canned reply.

Question 11:

Some workflows must start automatically the moment an event happens outside n8n — a form submission, a website chat message, or a call from another application — rather than on a schedule or manual run.

What is the primary purpose of a Webhook in n8n, and what node pairs with it to reply to the caller? (K11)

Suggestive answers (not exhaustive):

- A Webhook is a URL that external systems call to trigger the workflow in real time.
- Use cases: website chat, form submissions, payment/GitHub/Stripe events.
- Pair it with a Respond to Webhook node to send a reply back to the caller.
- Set Allowed Origins (CORS) = * so a browser page can call it.

Question 12:

n8n lets you connect different Large Language Models (LLMs) through their APIs, depending on the use case, performance needs and provider availability.

List three popular LLM providers commonly used with n8n for AI automation. (K12)

Suggestive answers (not exhaustive):

- OpenAI (GPT models)
- Google Gemini
- Anthropic Claude
- (others are available via API / OpenRouter)